SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I S.HARISH/192521003 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1 — AI-Powered Emergency Medicine Delivery Drones

Scenario:A government deploysAI-powereddrones todeliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

(i) How Greedy Best-First Search Behaves in This Scenario

Greedy Best-First Search (GBFS) is an informed search algorithm that selects the next node to explore based entirely on the heuristic function h(n). The heuristic h(n) estimates the cost to reach the goal from node n, and in this scenario it is taken as the straight-line (Euclidean) distance from the drone's current position to the destination delivery point.

Evaluation function: f(n) = h(n)

Here, f(n) is the evaluation score assigned to node n, and h(n) is the heuristic estimate (straight-line distance to goal). The algorithm always expands the node in the open list that has the lowest h(n) value, ignoring how much path cost has already been accumulated to reach that node.

How It Works Step by Step

1. The drone starts at its current position (the start node S). All immediate neighbours are added to an open list with their h(n) values.

2. The node with the smallest h(n) is selected and expanded next.

3. This greedy selection continues until the goal G (the medicine delivery point) is reached or no nodes remain.

(ii) How A* HandlestheSameSituation

A* is an informed search algorithm that selects the next node based on the combined evaluation function:

f(n) = g(n) + h(n)

where:

- g(n) is the actual path cost accumulated from the start node to node n. In this scenario, g(n) includes movement costs that vary with terrain type (flooded fields, strong winds, debris-filled areas).
- h(n) is the heuristic estimate of the remaining cost from n to the goal (straight-line distance in this case).
- f(n) is the estimated total cost of the cheapest path through n.

A* expands the node

with the lowest f(n) first. Because it accounts for both the cost already paid (g(n)) and the estimated future cost (h(n)), it balances progress toward the goal against the actual resources consumed so far.

How A* Balances Cost and Heuristic

Consider two candidate nodes: Node X is close to the goal (small h(n)) but required passing through a heavily flooded, high-cost segment (large g(n)). Node Y is slightly further from the goal (larger h(n)) but was reached via a low-cost clear route (small g(n)). GBFS would always choose X because it only looks at h(n). A* evaluates f(n) = g(n) + h(n) for both nodes and may choose Y if its total estimated cost is lower. This balance prevents A* from being tricked by short-looking but expensive paths.

When the heuristic h(n) is admissible (it never overestimates the true remaining cost), A* is guaranteed to find the optimal (lowest actual cost) path from start to goal. Straight-line distance is always an admissible heuristic in a physical environment because the actual travel distance can never be less than the straight-line distance.

Handling Variable Movement Costs

Edge costs in the drone's route graph represent movement costs between adjacent waypoints. These costs change with terrain (open air, flooded urban area, strong headwind zone) and weather. A* natively supports variable edge costs through the g(n) calculation: every time a new node is reached, g(n) = g(parent) + edge_cost(parent, n). If weather worsens a particular segment, its edge cost increases, and A* will prefer alternate routes with lower total f(n).

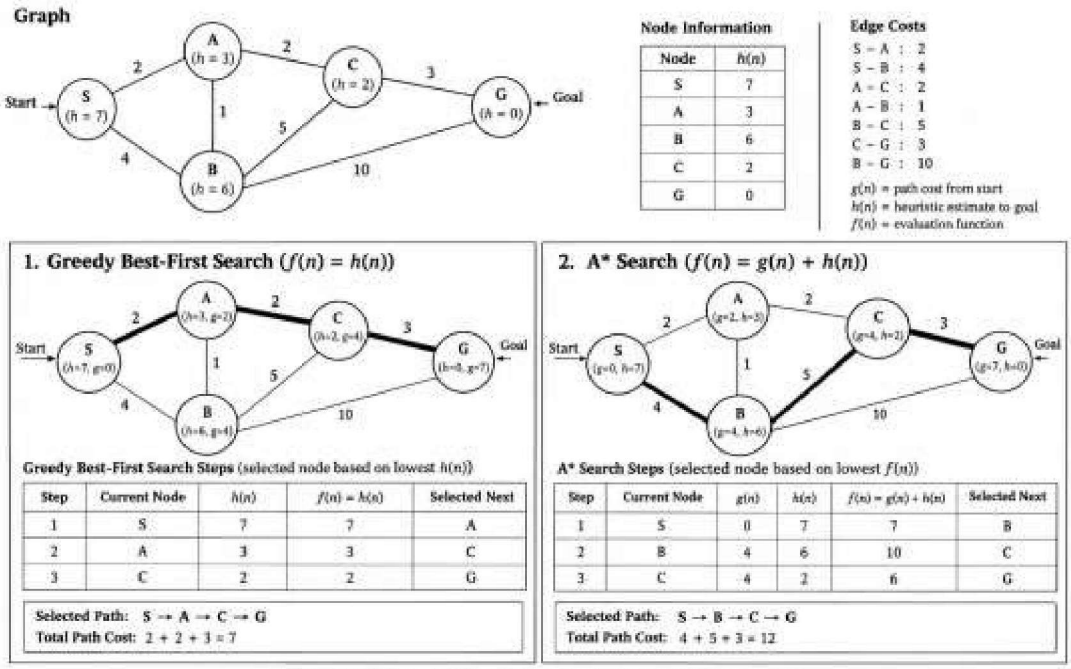


Figure 1: Comparison of Greedy Best-First Search and A* Search.

Figure 1: Comparison of Greedy Best-First Search and A* Search.

(iii) Impact ofHeuristicAccuracyon BothAlgorithms

The heuristic h(n) guides both algorithms toward the goal. Its accuracy has a major effect on how well each algorithm performs.

For Greedy Best-First Search: Since f(n) = h(n), the entire search depends on the heuristic alone. A highly accurate heuristic makes GBFS nearly as good as an optimal algorithm in practice. A poor heuristic (one that poorly estimates remaining distance, e.g., due to large flooded detours) will cause GBFS to make wrong turns and arrive at blocked or costly paths. Because GBFS never checks actual cost, it has no way to self-correct.

(iv) How Dynamic Changes (Blocked Paths) Affect Search Performance

In a flood-affected region, previously navigable paths can become blocked at any time due to rising water levels, debris, or strong winds. This makes the environment non-static during the search.

Effect on Greedy Best-First Search: If the path GBFS chose becomes blocked after the drone has already committed to a direction, GBFS must discard its current path and search again from the current position. Because GBFS is fast but not cost-aware, replanning may again lead to a suboptimal detour that is geographically close to the goal but physically dangerous.

(v) Recommendation: Most Suitable Algorithm

For the AI-powered emergency drone delivering medicine in a flood-affected region, A* Search with dynamic replanning capability is the recommended approach.

Justification:

- Minimum delay and optimality: A* with an admissible heuristic finds the path with the lowest total movement cost, accounting for variable terrain and weather conditions. GBFS may be faster in simple environments but can pick unnecessarily long or risky paths when the environment is complex.
- Safety: By incorporating edge costs that reflect risk (flooded terrain, high wind zones), A* naturally favours lower-risk paths. GBFS, which ignores accumulated cost, cannot distinguish between a safe cheap route and a dangerous cheap-looking route.

Question 2 — Smart-City Traffic Signal Optimization

Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions.

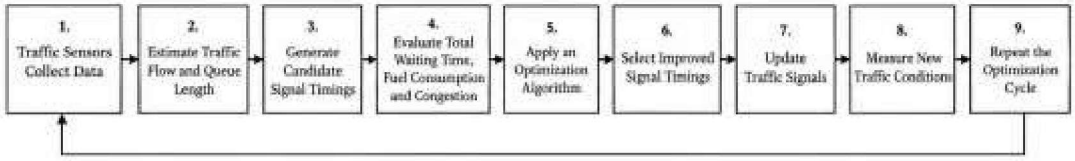
Formulation as an Optimization Problem

Traffic signal optimization can be formally stated as an optimization problem with the following components:

Decision Variables: For each intersection i (i = 1, 2, ..., N) and each signal phase j, the decision variable is t_{i,j}, the green-phase duration assigned to phase j at intersection i. The set of all t_{i,j} values forms the state (solution) that the optimizer must choose.

Objective Function: The goal is to minimize a combined cost function that captures the three performance measures:

Minimize F(T) = w1 * W(T) + w2 * C(T) + w3 * K(T)



Comparison of Optimization Algorithms		
Hill Climbing	**Simulated Annealing**	**Genetic Algorithms**
Can become trapped in local optima.	Can sometimes accept worse intermediate solutions.	Evolve a population of candidate solutions.

Figure 2: AI-Based Traffic Signal Optimization Process.

Figure 2: AI-Based Traffic Signal Optimization Process.

(i) Hill Climbing in This Scenario and Its Limitations

Hill Climbing is a local search algorithm that starts with a randomly or heuristically selected initial solution and iteratively moves to a neighbouring solution if it improves the objective value. In the traffic context, a neighbour of a current timing plan T is a new plan T' obtained by slightly adjusting one or more green-phase durations (for example, adding 5 seconds to one phase and removing 5 seconds from another to maintain the cycle length). If $F(T') < F(T)$, the algorithm moves to T'. It continues until no improvement can be found among the neighbours of the current state.

Advantages: Hill Climbing is computationally cheap and easy to implement. It can quickly move away from a bad initial state toward a better region of the search space.

Limitations in Traffic Optimization:

(ii) Local Maxima, Plateaus, and Ridges

Local Maximum (in minimisation: local minimum): A state from which every small change worsens the objective. The algorithm incorrectly concludes it has found the best solution. Example: a timing plan that works well for morning peak-hour traffic but is not globally optimal for all-day traffic patterns.

Plateau: A flat region where neighbours have equal or nearly equal objective values. The algorithm makes no progress. Example: many timing plans that differ by a few seconds produce identical estimated waiting times because the sensor data resolution is low.

Ridge: A connected sequence of good solutions in a high-dimensional space that Hill Climbing cannot follow because it can only make moves in predefined directions. Example: optimal signal coordination along a one-way arterial requires specific phase offsets at each of many intersections simultaneously; changing one at a time always seems to worsen performance even though all changed together would help.

(iii) Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA):

SA is inspired by the physical annealing process in metallurgy. It extends Hill Climbing by allowing occasional uphill moves (moves that worsen the objective) with a probability that decreases over time.

Main Steps:

- Start with an initial solution T (for example, current deployed timing plan).
- Set an initial temperature parameter T_temp to a high value.
- At each step, generate a random neighbour T' by perturbing one or more signal timings.
- If F(T') < F(T) (improvement), always accept T'.
- If F(T') >= F(T) (worsening), accept T' with probability exp(-(F(T')-F(T)) / T_temp).
- Gradually reduce T_temp (cooling schedule) so that fewer bad moves are accepted over time.
- Continue until the temperature is near zero or a time limit is reached.

Comparison of Algorithms

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Local optima	Gets stuck	Can escape (probabilistic)	Avoids via population diversity
Scalability	Poor for large N	Moderate	Better; parallelisable
Optimality	Not guaranteed	Asymptotically (slow cool)	Not guaranteed but near-optimal
Online adaptation	Limited	Good (warm restart)	Moderate (generation time)
Computational cost	Low	Moderate	High per generation

(iv) Recommendation: Best Approach for Scalability and Adaptability

For a smart-city traffic system with hundreds of intersections and continuously changing traffic patterns, Genetic Algorithms — or a hybrid combining GA and Simulated Annealing — are recommended.

Justification:

- Scalability: GAs are naturally parallelisable. Each individual in the population can be evaluated independently on separate processors or machines, making them practical for large N.
- Avoidance of local optima: The population-based search explores multiple regions simultaneously. Crossover allows good timing sub-plans (e.g., a well-coordinated arterial segment found in one individual) to combine with good sub-plans for other segments found in other individuals.
- Continuous adaptation: The system can be designed as a rolling optimisation loop. As new traffic data arrives every few minutes, the GA is re-run with the current best solution as part of the initial population (seeding), ensuring that the new plan does not deviate drastically from the deployed plan while still improving on it.

Question 3 — Autonomous Mars Exploration Rover

Scenario: An autonomousrover is deployedon Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

(i) Why This Problem Requires an Online Search Agent

An offline search agent assumes that the agent has full knowledge of the environment before it begins to act. It builds a complete search tree or graph, finds the optimal plan, and then executes it without changing the plan. An offline approach is therefore only suitable in fully observable, static, and deterministic environments where all information is available in advance.

Mars exploration fails all of these conditions. The Martian surface has not been fully mapped at the resolution required for rover navigation. The terrain contains craters, loose rocks, sandy patches, and slopes that are only visible to the rover's sensors when it is physically close to them. Communication delays between Earth and Mars can exceed 20 minutes in each direction, so the rover cannot ask Earth controllers what to do at each step in real time. Furthermore, the terrain may shift (loose sand, rock slides) as the rover moves, changing the effective environment.

(ii) Challengesof PartialObservabilityand Dynamic Environments

Partial Observability: The rover cannot sense beyond its immediate camera and instrument range. At any point, a large fraction of the terrain ahead is unknown. The rover cannot know whether a path that appears open in the distance is actually traversable until it gets closer. Rocks hidden below a sand surface, for example, may only become apparent under the rover's wheels.

(iii) How the Rover Builds and Updates Its Internal Model

The rover maintains an internal occupancy map: a grid representation of the terrain in which each cell is marked as free (traversable), occupied (obstacle), or unknown. The rover starts with all cells unknown except those it can directly observe.

Cycle of Operation:

- Observe: The rover's cameras, LIDAR, and terrain sensors scan the immediate surroundings. Obstacles (rocks, craters, steep slopes) are detected and their positions recorded relative to the rover.
- Update Internal Map: Detected obstacles are marked in the occupancy grid. Free space that is visible and clear is also marked. Unknown cells remain until the rover gets close enough to observe them.
- Evaluate Available Actions: The rover considers the set of possible moves from its current position (forward, left, right, small turns). It filters out moves that would take it into occupied or dangerously uncertain cells.
- Select a Safe Action: A search or planning algorithm (such as A* on the current partial map, or a local path planner) selects the best available move toward the target sample collection point, subject to safety constraints.

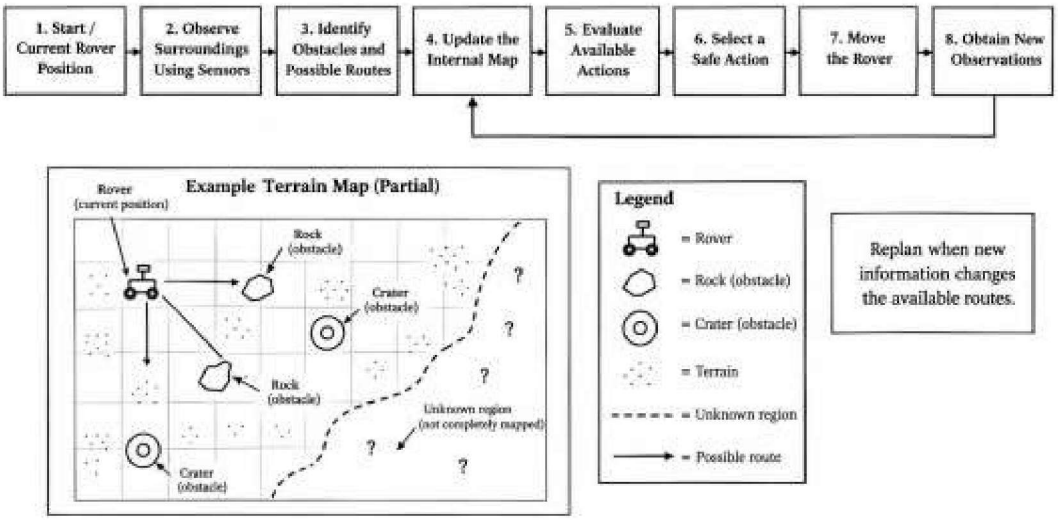


Figure 3: Online Search and Continuous Map Updating in a Mars Rover.

Figure 3: Online Search and Continuous Map Updating in a Mars Rover.

(iv) Comparison of Online Search with Uninformed and Informed Strategies Uninformed Search (Breadth-First Search, Depth-First Search, Uniform-Cost Search): These algorithms require a complete graph of the environment. They cannot operate when most of the graph is unknown. They are appropriate for problems where the full state space is known in advance (e.g., puzzle solving, known-map navigation). On Mars, the rover has no complete graph to feed into these algorithms.

Informed Search (A*, Greedy Best-First Search): These algorithms use a heuristic to guide search and require at least a partial map. A* is frequently used in rover systems for navigation over the known portion of the terrain.

(v) Recommended Approach

The recommended approach for the Mars rover is online search with dynamic replanning, specifically a variant of D* Lite or a similar incremental online planner, combined with conservative safety constraints.

Justification:

- Adaptability: D* Lite efficiently repairs the route when new obstacles are seen, without restarting the entire search. This is essential because the Mars surface will regularly reveal new hazards as the rover advances.
- Uncertainty handling: The system does not require a complete map. It operates correctly with a partial, growing map and maintains a safety buffer around uncertain cells (treating unknown cells as potentially occupied until confirmed safe).
- Computational limits: The rover's onboard computer has limited processing power. D* Lite and LRTA* are computationally efficient because they repair existing plans incrementally rather than solving the full problem from scratch on every update.

Question 4 — University Examination Timetable Using CSP

Scenario: A universityis designing anAI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Formulation as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem consists of three components: a set of variables, a domain for each variable, and a set of constraints that restrict the combinations of values the variables may take. The timetabling problem maps naturally onto this structure.

(i)Variables,Domains,and Constraints

Variables: Each variable represents an exam assignment decision. Let E = {E1, E2, ..., En} be the set of exams that must be scheduled. Each variable Ei represents the assignment of exam i to a (time slot, hall) pair. The value assigned to each variable specifies when and where the exam takes place.

Domains: The domain of each variable Ei is the set of all (time slot, hall) combinations that are initially possible for that exam. Time slots may be defined as specific date-and-hour blocks (for example, Monday 9–11 AM, Monday 2–4 PM, Tuesday 9–11 AM). Available halls are the university's exam rooms. If there are T time slots and H available halls, the initial domain of each exam variable has up to T × H values. In practice, the domain is immediately restricted by obvious constraints such as hall availability (a hall under maintenance is removed from domains) and known scheduling windows.

Constraints:

- No student overlap (Hard): Two exams that share at least one enrolled student must not be assigned the same time slot. This is the most fundamental constraint. It is a hard constraint because there is no flexibility: a student cannot be in two places at once. If exams Ei and Ej share students, then value(Ei).timeslot ≠ value(Ej).timeslot.
- Hall capacity (Hard): The number of students registered for exam Ei must not exceed the seating capacity of the hall assigned to it: enrolled(Ei) ≤ capacity(value(Ei).hall).
- Hall availability (Hard): A hall that is unavailable at a particular time (due to maintenance, bookings, or another exam) cannot be assigned to any exam during that period: value(Ei).hall must be available during value(Ei).timeslot.

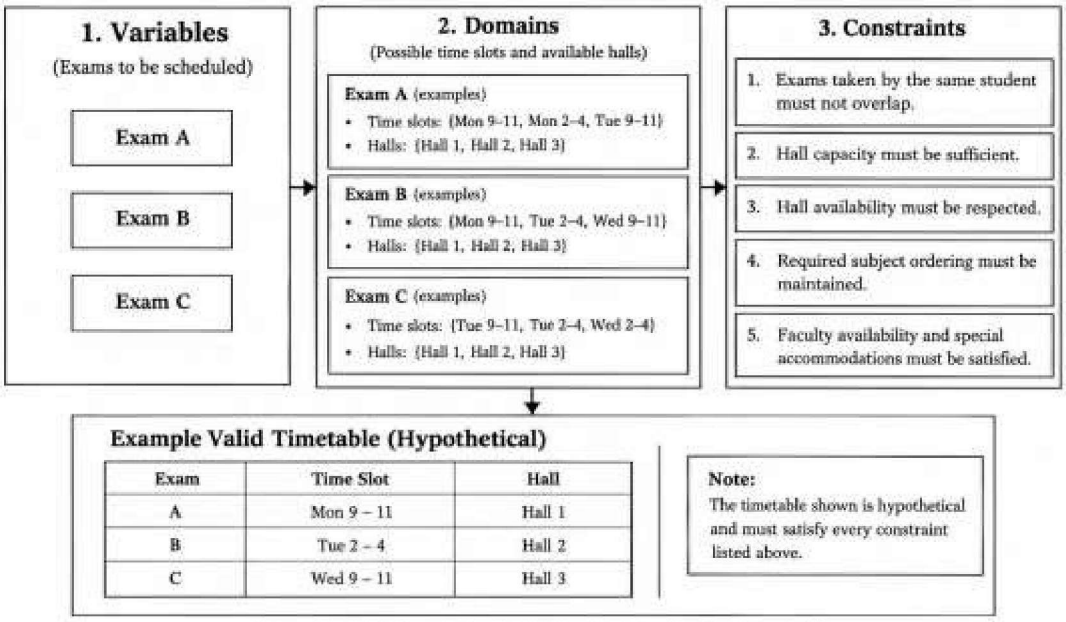


Figure 4: CSP Formulation for University Examination Timetabling.

Figure 4: CSP Formulation for University Examination Timetabling.

(ii) How Backtracking Search Works in This Scenario

Backtracking Search is the standard algorithm for solving CSPs. It assigns values to variables one at a time, checking constraints after each assignment. If a contradiction is found, it undoes the last assignment and tries a different value.

Step-by-Step Process:

- Select an unassigned variable: Pick the next exam Ei that has not yet been assigned a (time slot, hall) pair. Initially, this is any exam; heuristics guide this choice (discussed below).
- Order the domain values: Choose an ordering of the values in D(Ei) (the current domain of Ei). The algorithm will try these values in order.
- Assign a value: Tentatively assign the first value (ts, h) from D(Ei) to Ei.
- Check constraints: Check whether the assignment of Ei = (ts, h) violates any constraint with already-assigned variables. For example: check that no other already-scheduled exam in the same time slot ts has students in common with Ei; check that hall h has sufficient capacity; check that hall h is available at time ts; check any ordering constraints with already-assigned exams.
- If consistent: Proceed to the next unassigned variable recursively.

(iii) Forward Checking and Constraint Propagation

Plain backtracking only detects violations after they occur. Forward Checking (FC) improves efficiency by eliminating values from future variable domains immediately after each assignment, preventing many dead-ends before they are reached.

How Forward Checking Works: When a variable Ei is assigned value (ts, h), the algorithm looks ahead at each unassigned variable Ej that shares a constraint with Ei. It removes from D(Ej) any value that would violate that constraint given Ei's assignment. If D(Ej) becomes

empty, the algorithm immediately backtracks without exploring further (since no valid assignment for Ej exists).

(iv) Real-World Challenges and Recommended Strategy for Scalability

Real-World Challenges:

- Large scale: A university may have hundreds of courses, thousands of students, dozens of halls, and dozens of time slots. The domain of each variable (time slot × hall combinations) is large, and the number of constraints is quadratic in the number of exams sharing students.

- Conflicting constraints: Faculty availability, special accommodation requirements, and ordering constraints may combine to make some exams very tightly constrained, leaving few valid assignments.
- Soft preferences: Fairness (spreading exams across the week) and convenience (not scheduling consecutive exams for the same students) are important but hard to encode as hard constraints without making the CSP infeasible.

Question 5 — Strategic Game AI Using Minimax and Alpha-Beta Pruning

Scenario: Agamingcompanyis developing an AI opponent fora real-time strategy game.The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

(i)How theMinimaxAlgorithm ModelsThisProblem

Minimax is a decision-making algorithm for two-player zero-sum games. In a zero-sum game, one player's gain is exactly the other player's loss. In the strategy game, the AI (MAX player) tries to maximise its evaluation score, and the human opponent (MIN player) tries to minimise it. The game state at any point is represented as a node in a game tree.

Structure of the Game Tree:

- The root node represents the current game state.
- Each level (ply) of the tree alternates between MAX moves and MIN moves. At MAX nodes (AI's turn), the AI selects the move that leads to the highest-valued child. At MIN nodes (opponent's turn), the opponent selects the move that leads to the lowest-valued child.
- Leaf nodes (at the bottom of the tree, or when the game ends) are assigned a score by an evaluation function. A positive score favours the AI; a negative score favours the opponent.

Minimax Procedure: At a MAX node, return the maximum of the children's values. At a MIN node, return the minimum of the children's values. At a leaf, return the evaluation function value. The AI selects the root-level move that leads to the child with the maximum minimax value, knowing that the opponent will always respond optimally (minimise).

This models rational play: the AI prepares for the best response the opponent could make, rather than assuming the opponent will make mistakes.

(ii) Computational Challenges of Large Game Trees

Branching Factor and Tree Size: If the game has a branching factor b (number of legal moves per state) and the search goes to depth d, the total number of nodes examined is b^d. For a strategy game with b = 30 possible moves per turn and a desired search depth of d = 6 plies, this is 30^6 = 729,000,000 nodes. With d = 8, it becomes 30^8 ≈ 656 billion nodes. Examining all these nodes at each move decision is computationally infeasible within a real-time budget. Time Limits: The AI must make a decision within a fixed time (for example, 100–500 milliseconds per move in a fast-paced strategy game). Exhaustive minimax at depth 6 or beyond cannot be completed in this time for a branching factor of 30 or more.

Memory: Storing the entire game tree in memory is infeasible for deep searches. Algorithms must process the tree incrementally.

Horizon Effect: Cutting off the search at a fixed depth means the AI does not see consequences beyond that depth. An evaluation function must approximate the value of non-terminal states, introducing inaccuracy.

(iii) How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta Pruning is an optimisation of minimax that avoids evaluating subtrees that cannot influence the final decision. It maintains two bounds:

- Alpha (α): The best (highest) value that the MAX player is guaranteed to achieve so far along the current path from the root. Initially $\alpha = -\infty$.
- Beta (β): The best (lowest) value that the MIN player is guaranteed to achieve so far along the current path from the root. Initially $\beta = +\infty$.

Pruning Condition: $\alpha \geq \beta$

During a MIN node evaluation, if the current value of that node falls to or below α (the best the MAX player can already guarantee elsewhere), the MAX player will never choose this path. The remaining children of this MIN node are pruned (not explored). Similarly, during a MAX node evaluation, if the current value rises to or above β, the MIN player will never allow this path, and remaining children of this MAX node are pruned.

How Bounds Are Updated:

- When a MAX node finds a child value v, update $\alpha = \max(\alpha, v)$. If $\alpha \geq \beta$ at any point, prune remaining children of the current MIN ancestor.
- When a MIN node finds a child value v, update $\beta = \min(\beta, v)$. If $\alpha \geq \beta$ at any point, prune remaining children of the current MAX ancestor.

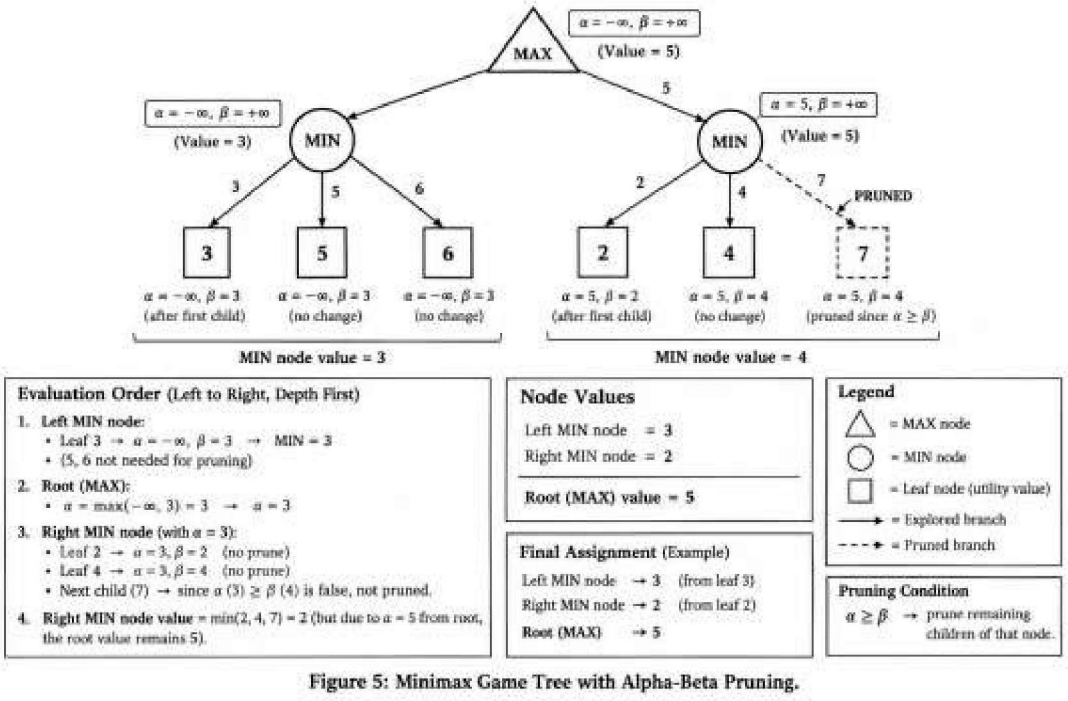


Figure 5: Minimax Game Tree with Alpha-Beta Pruning.

Figure 5: Minimax Game Tree with Alpha-Beta Pruning.

(iv) Design of an Evaluation Function

When minimax reaches its depth limit or a non-terminal leaf, it calls an evaluation function E(s) to score the game state s. The evaluation function must efficiently approximate the game-theoretic value of s without searching further. For a strategic game, a weighted linear combination of game-relevant features is commonly used:

$E(s) = w1*R(s) + w2*U(s) + w3*T(s) + w4*O(s) + w5*D(s) + w6*I(s)$

Factors and their meaning:

Factor	Symbol	Meaning
Resource advantage	R(s)	AI resources minus opponent resources (gold, energy, troops). More resources enable future actions.
Unit strength	U(s)	Sum of AI unit combat strength minus opponent's. Directly affects battle outcomes.
Territory control	T(s)	Proportion of key map areas controlled by AI. Territory often provides resource income and strategic advantages.
Objective progress	O(s)	Progress toward mission objectives (e.g., base construction, flag capture). Directly tied to winning conditions.
Defensive position	D(s)	Quality of AI's defensive formation and fortifications. Reduces vulnerability to opponent attacks.
Immediate threats	I(s)	Measure of active threats to AI units or structures. Negative term: high threat worsens score.

(v)RecommendedStrategyforBalancingOptimalityand Real-TimePerformance

For a real-time strategy game with strict time limits, the recommended approach combines Alpha-Beta Pruning, iterative deepening, and move ordering within a time budget.

Iterative Deepening: Instead of committing to a fixed search depth, the AI performs a complete alpha-beta search to depth 1, then depth 2, then depth 3, and so on. At each depth it records the best move found so far. When the time budget expires, the AI plays the best move found in the most recently completed search. This guarantees that the AI always has a move available even if time runs out, while making the best use of available time.

Move Ordering: Alpha-Beta Pruning is most effective when the best moves are examined first, because they quickly update α and β, enabling more pruning.